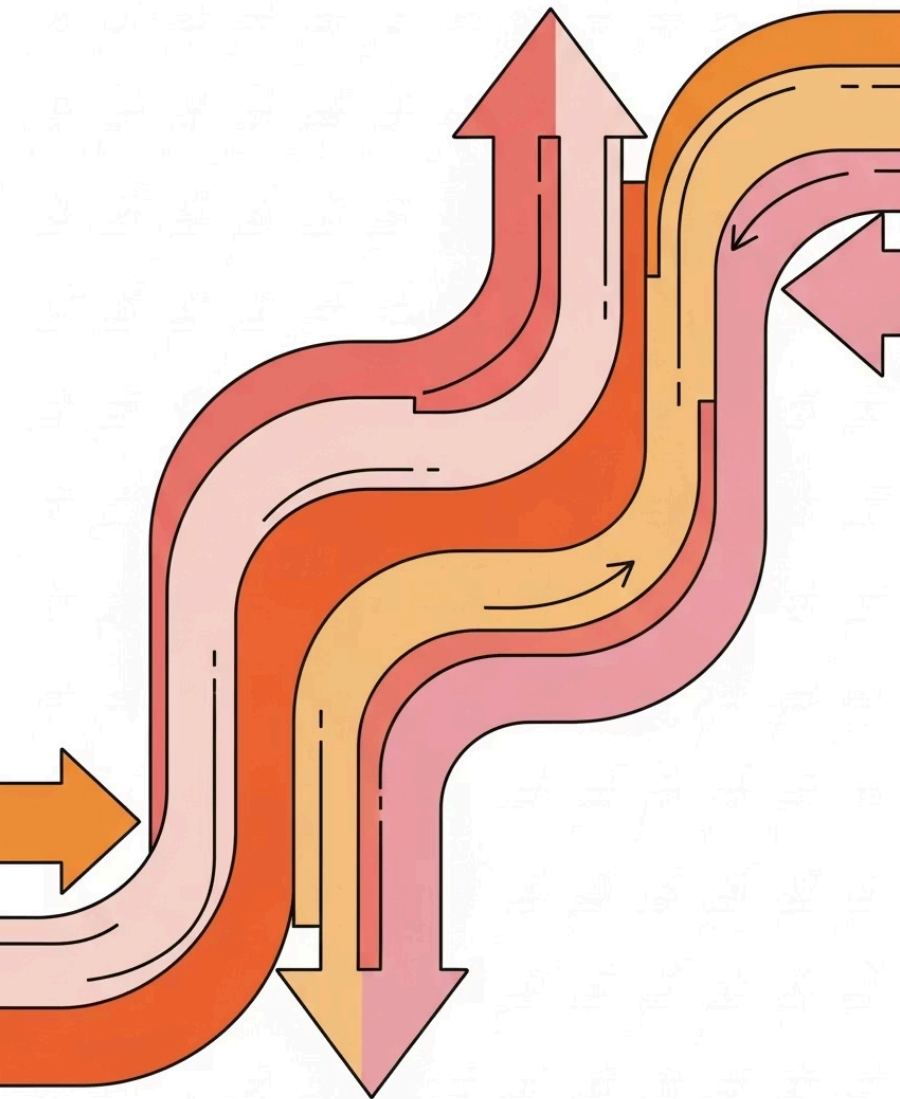




Lambdas: la base de la programación funcional en Java

Funciones como datos en Java

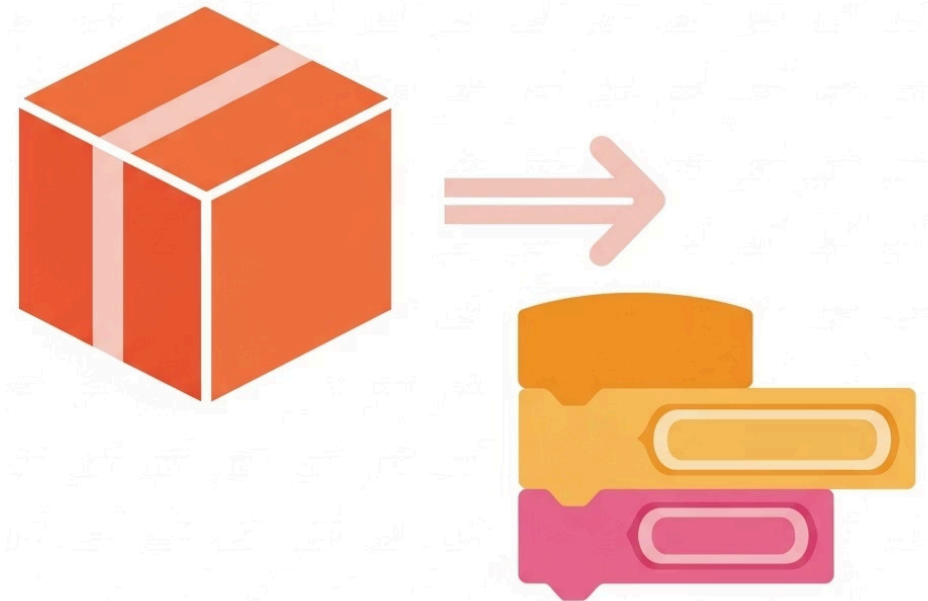
La pieza que hace posible que los Streams sean tan compactos.

¿Qué es una expresión lambda?

Definición simple

Una lambda es una forma corta de escribir una función anónima: código que podemos pasar como dato.

- No tiene nombre
- Puede recibir parámetros
- Puede devolver un valor



Las lambdas son la puerta de entrada a la programación funcional en Java, permitiéndonos tratar el comportamiento (código) como datos.

Antes de Java 8 vs con lambdas

La evolución hacia un código más conciso y expresivo

Antes (Java 7)

```
Collections.sort(nombres, new Comparator() {  
    @Override  
    public int compare(String a, String b) {  
        return a.compareTo(b);  
    }  
});
```

Clase anónima con mucho código repetitivo

Con Java 8+

```
Collections.sort(nombres,  
    (a, b) -> a.compareTo(b));
```

Lambda clara y concisa, enfocada solo en lo importante

¿Ves la diferencia? Pasamos de 5 líneas de código a solo 1, manteniendo la misma funcionalidad.

Estructura de una lambda

1

Parámetros

Variables de entrada entre paréntesis. Si solo hay un parámetro, los paréntesis son opcionales.

Ejemplos: (x), (a, b), x, ()

2

Flecha

El operador `->` que separa los parámetros del cuerpo de la función.

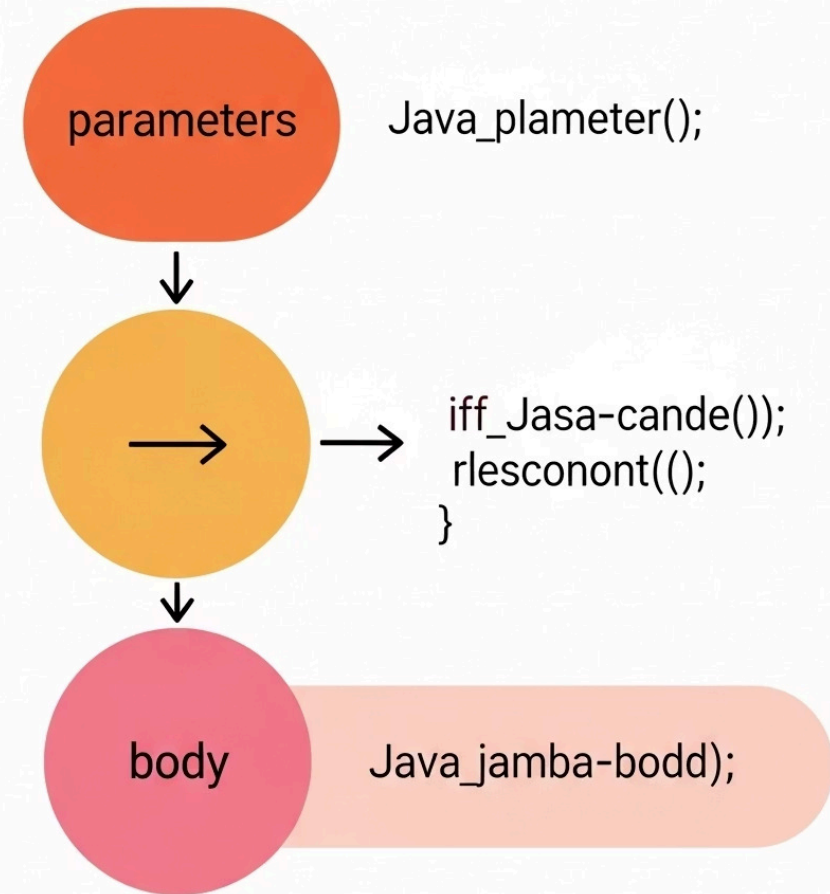
3

Cuerpo

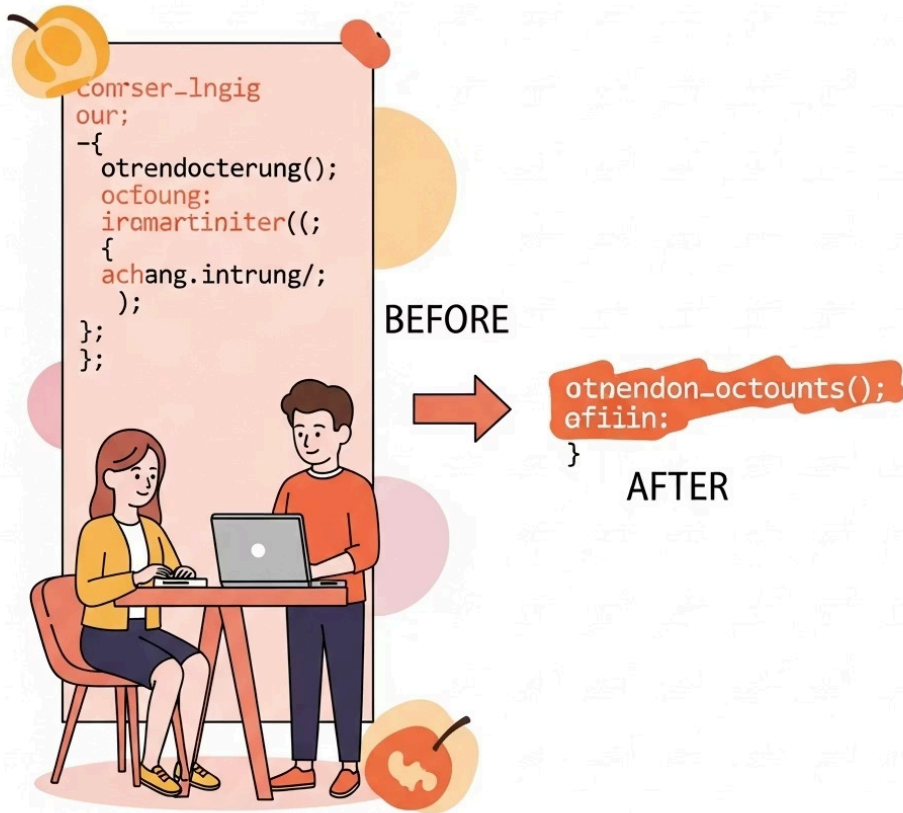
El código a ejecutar. Para expresiones simples no necesita llaves ni `return`.

Ejemplos: `x * 2`, `a + b`, `{ return x > 0; }`

Lambda syntax



Ventajas de las lambdas



Menos código repetitivo

Elimina la necesidad de escribir clases anónimas completas para implementar interfaces funcionales.

Más legible y expresivo

Permite centrarse en la lógica importante sin distracciones sintácticas.

Integración natural con Streams

Las lambdas son el complemento perfecto para las operaciones de flujo en colecciones.

Programación funcional

Abre la puerta a un estilo de programación basado en funciones y transformaciones.

Lambdas y Streams

El dúo perfecto para procesamiento de datos

Las lambdas definen **qué hacer** en cada paso del Stream, mientras que el Stream define **cómo hacerlo** de manera eficiente.

- ❗ Los Streams procesan los datos de forma perezosa (lazy), ejecutando las operaciones solo cuando es necesario, lo que mejora el rendimiento.

Lo que aprendimos hoy

Qué es una expresión lambda

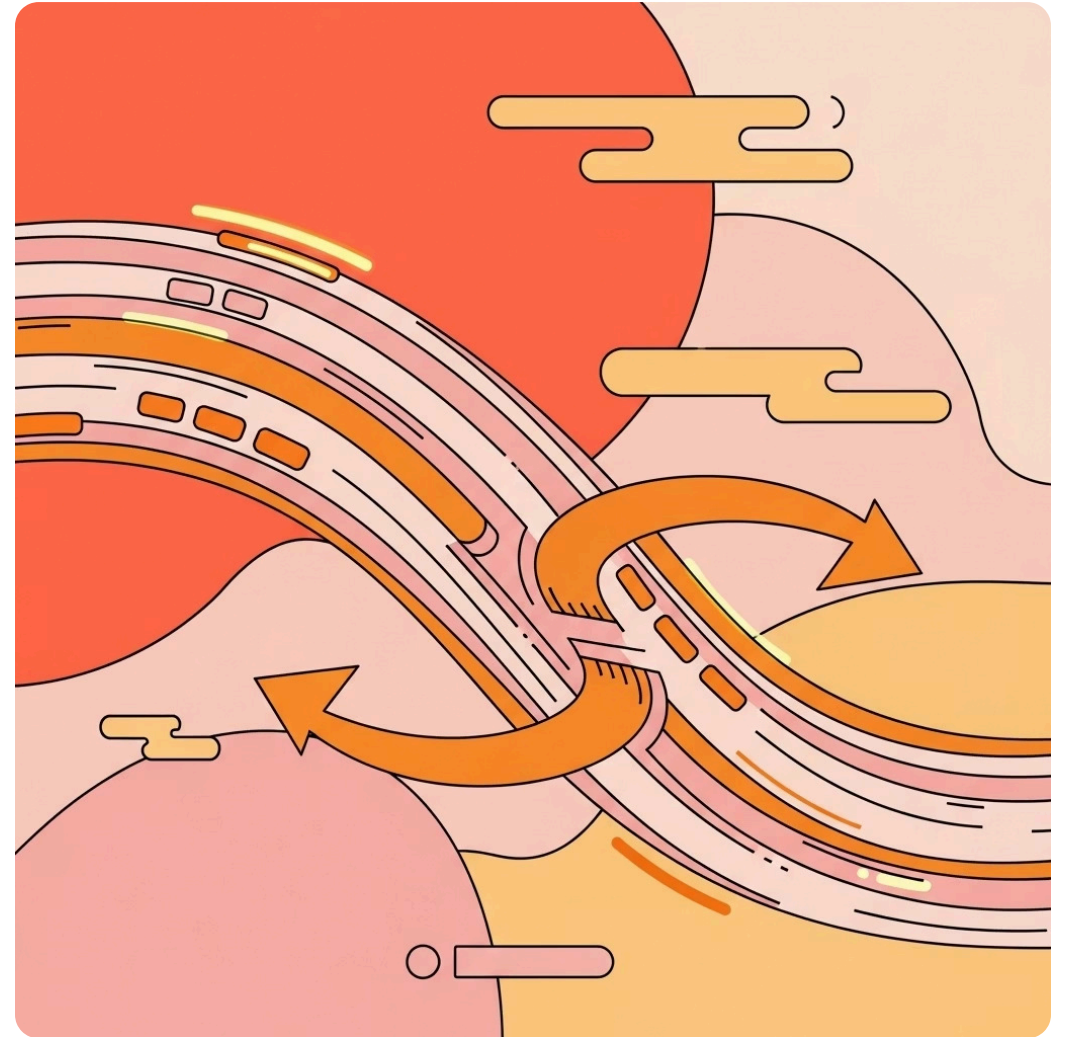
Una función anónima que podemos pasar como dato

Cómo se escribe

Parámetros, flecha y cuerpo: `(param) -> expresión`

Por qué es clave para los Streams

Permite definir operaciones de forma concisa



En el próximo video veremos operaciones intermedias en Streams, para transformar y filtrar datos de forma funcional.